



Programación Segura

Conceptos de seguridad

Contacto: zS25022557@estudiantes.uv.mx



- Ingresen a la página:

https://sucuri.net/guides/owasp_top_10_2021_edition/

- ¿Qué son los ataques de inyección?
- ¿Qué son los ataques de autenticación rota?
- Identifiquen que es lo que la mayoría de los ataques tienen en común
- Identifiquen al menos un caso real por cada vulnerabilidad
- ¿Cómo se pueden prevenir?

¿Qué son las Tendencias de Ciberseguridad?



- Son los cambios en la forma en que ocurren los ataques y cómo la tecnología evoluciona para enfrentarlos. No son modas, sino respuestas a nuevas formas de vulnerar los sistemas

¿Por qué importan?

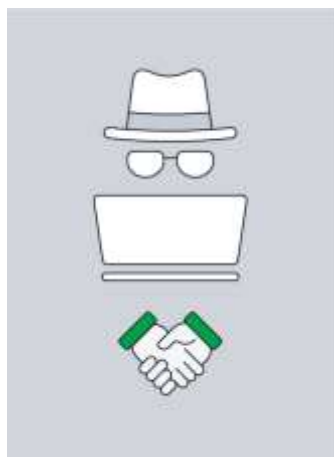
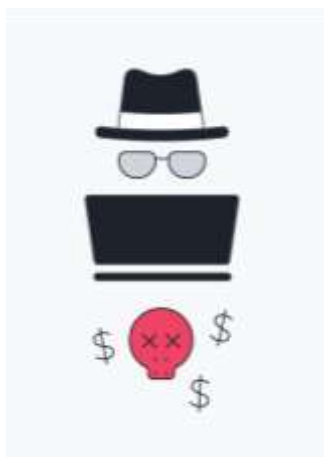
- Nos enseñan que la seguridad no es estática, los métodos que funcionaban hace pocos años ya no son suficientes frente a ciberdelincuentes más organizados y automatizados



- Es una preocupación constante para las organizaciones y empresas, ya que deben resistir los ataques mientras que siguen operando y cuidan los datos privados

¿Quiénes llevan a cabo estos ataques?

**Equipo
Rojo**





¿Porqué esto es tan peligroso el hackeo actualmente?

- Existe una tendencia hacía el Internet de las cosas, donde con el paso del tiempo las vulnerabilidades de seguridad puedan ser una verdadera amenaza directa para la vida e integridad de las personas
- Las tendencias de nuevos escenarios de ciberterrorismo incrementan y no solo las empresas pueden peligrar
- Por esto mismo se necesita gente preparada que pueda defender estas amenazas (El equipo azul)



- El equipo azul no es muy popular y tantos medios de información como las redes sociales los hacen ver como los perdedores generalmente
- La industria de la seguridad en la informática tiende a tener un periodo de desempleo negativo y esto se debe a que existen más puestos disponibles que personal que pueda cubrirlos
- Contratar personal es difícil, pero contratar buenas personas es todavía más difícil

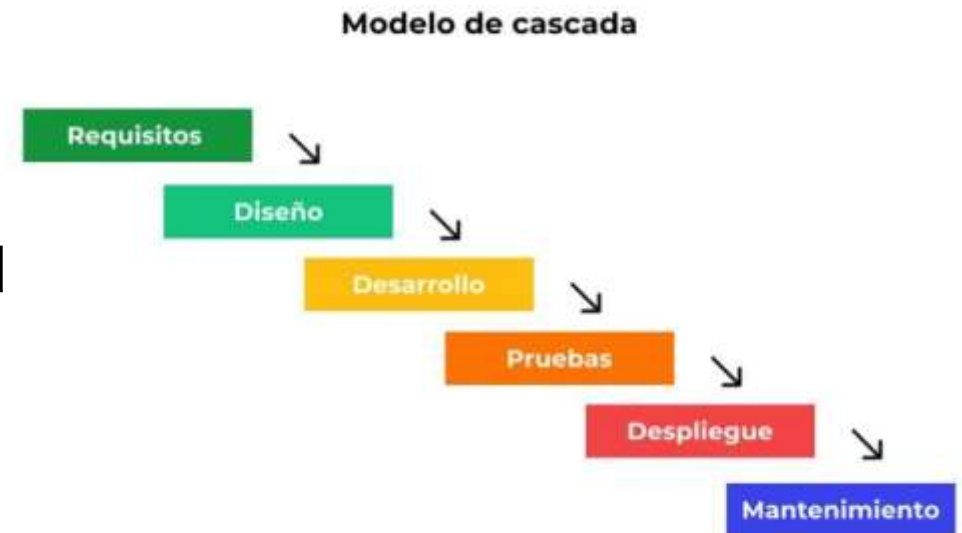


- La seguridad involucra una gran cantidad de elementos y cosas a considerar
- Desde la infraestructura física de red hasta la programación misma de los servicios



- Un **plan de seguridad completo** involucra los siguientes aspectos generales:

- Sentar bases
- Establecer equipos
- Establecer una postura base de seguridad
- Evaluación de amenazas y riesgos
- Priorización
- Creación de hitos
- Realizar simulacros y ejercicios basados en casos
- Expansión y mejora continua de los equipos de seguridad





- **Sentar bases**

- Define que se va a hacer y como se va a hacer
- Establece el rumbo del proyecto
- Define reglas principales y acuerdos que todo el equipo debe respetar desde el primer día





- **Establecer equipos**
 - Se **asignan responsabilidades** claras para saber quién se encargará de **revisar la seguridad, documentación, base de datos, hablar con el cliente** en caso de problemas o nuevos requisitos y **quién responderá si algo falla**, evitando que sea tarea de una sola persona.





- Establecer una postura base de seguridad
 - Definir el **mínimo obligatorio** de protección que debe tener tu proyecto o aplicación antes de que empiece a crecer.



Acceso Controlado a Usuarios.

*Usuario:

*Nueva Contraseña:

*Repetir Contraseña:



- **Evaluación de amenazas y riesgos**

- Se analiza qué le puede fallar al software o a la infraestructura y cuáles son los puntos débiles a los que se enfrenta el proyecto.
- Debes detenerte a pensar de forma organizada: **¿Qué nos podría salir mal?, ¿por dónde nos podrían atacar? y ¿qué tan grave sería si pasa?**





- **Priorización**

- Como el tiempo y los recursos son limitados, se ordenan los problemas encontrados para decidir qué se debe solucionar primero y justificar por qué.

A)



B)

✖ Form not fully sent. Error communicating with 1 of 1 storage services.

Name *

test	test
First Name	Last Name

Email Address *

C)

Log In

New to this site? [Sign Up](#)

Email

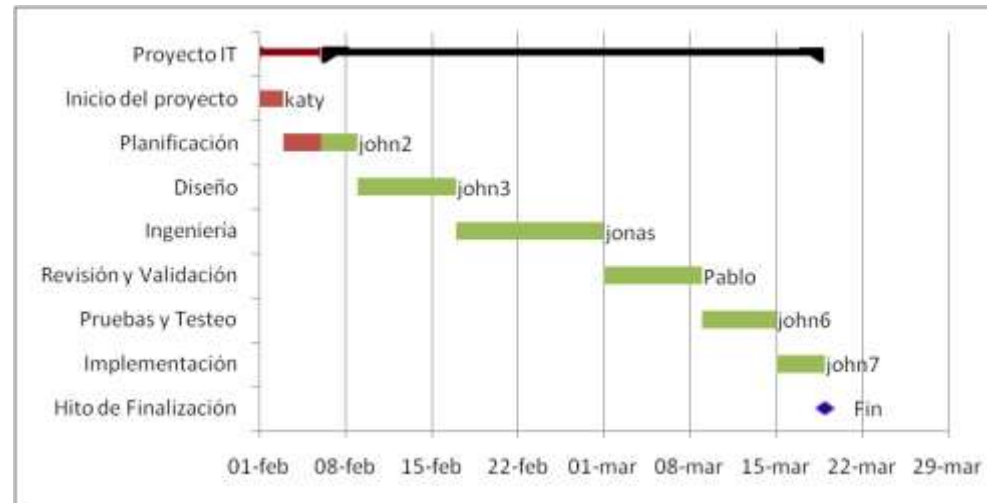
Password

[Forgot password?](#)



- Creación de hitos

- Se establecen **fechas límite y metas claras** en el calendario del proyecto para medir el avance de la seguridad.





- Realizar simulacros y ejercicios basados en casos
 - Poner a prueba las **defensas de tu sistema** y la **capacidad de reacción del equipo** mediante **pruebas prácticas** que **imitan situaciones de riesgo reales**, pero en un entorno controlado y seguro.

Nombre *

Fecha de nacimiento *

 Utiliza un formato que coincida con el solicitado
Formato DD/MM/AAAA





- **Expansión y mejora continua de los equipos de seguridad**
 - **Entender que la ciberseguridad nunca es un proyecto que se "termina" y se archiva, sino un proceso de aprendizaje y crecimiento constante que debe evolucionar a la par de la tecnología y de las personas.**





- Imagina que te han asignado a liderar la parte de seguridad para el desarrollo de un nuevo sistema de software de una empresa (A tú elección).
- Basándote en los conceptos vistos en clase estructura un plan de seguridad y responde las siguientes preguntas:
 - Si pudieras elegir una sola regla obligatoria para todo el equipo de desarrollo, ¿Cual sería y por qué?
 - Si el tiempo fuera limitado y hubiese múltiples fallas, ¿Cómo decidirías que problema de seguridad atender primero?

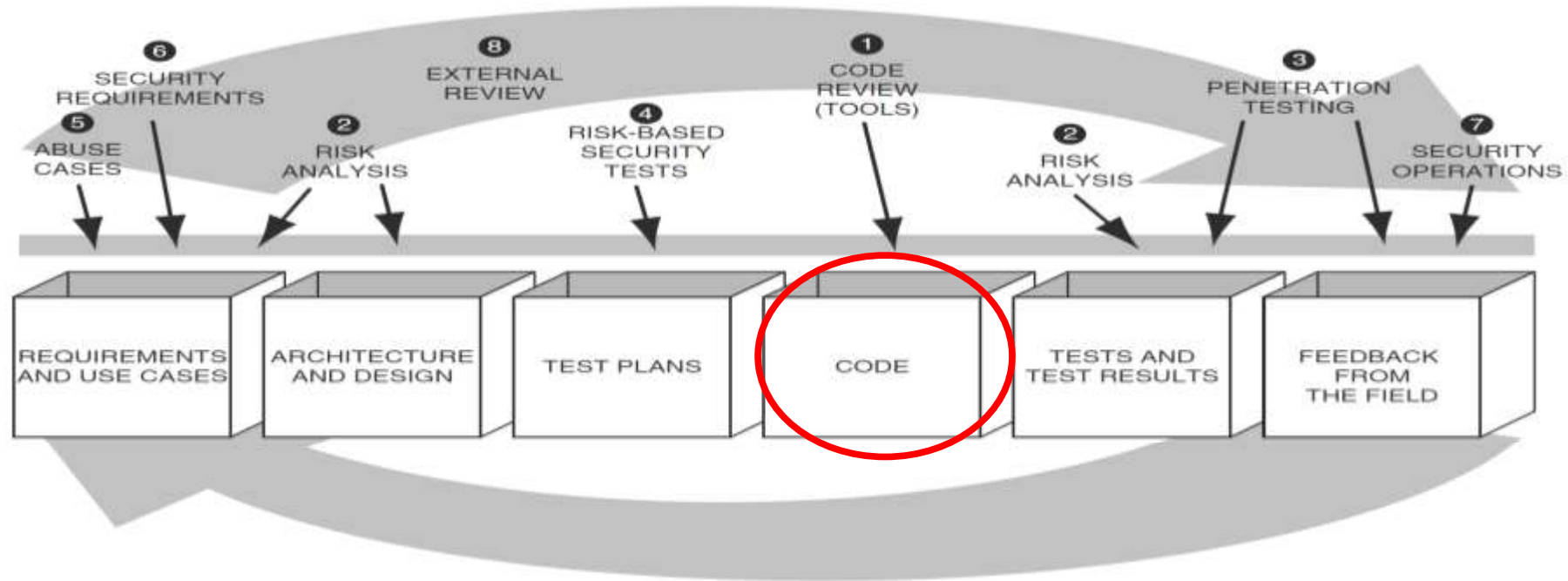
PLAN DE SEGURIDAD

1. Sentar bases
2. Establecer equipos
3. Establecer una postura base de seguridad
4. Evaluación de amenazas y riesgos
5. Priorización
6. Creación de hitos
7. Realizar simulacros y ejercicios basados en casos
8. Expansión y mejora continua de los equipos de seguridad

¿Dónde estoy yo?



- Un licenciado en Ingeniería ciberseguridad e Infraestructura de computo debe conocer todos los aspectos de seguridad.



¿Dónde estoy yo?



- Si bien, el **enfoque de la carrera no es el desarrollo de software, se debe ser capaz de identificar problemas de seguridad en el código y entender el mismo** para poder participar en la toma de decisiones respecto a la mejora de la calidad de la seguridad
- **Las actividades de todo un conjunto de personas pueden verse afectadas por los problemas de seguridad**, el perfil que tienen se orienta a mantener los servicios operando en todo momento y tener siempre la información crítica protegida

El problema de la seguridad del software



- Siempre se ha pensado que la mejor manera de hacer software seguro es estudiar los errores del pasado y asegurarnos de no repetirlos.
- Si vemos la lista de vulnerabilidades de OWASP
¿Creen que se esta aprendiendo de los errores?
- ¿Dónde se encuentra el problema?



- Algunas personas suelen creer que la seguridad es un problema del cual se tienen que preocupar únicamente a unas cuantas personas del equipo de trabajo (Ciberseguridad o a los programadores).
- Sin embargo, es que esto le debería de preocupar a todos.



- La forma en la que se desarrolla el software esta muy relacionado a que sea útil, entendible y funcional.
- La seguridad no tiene una atención mayor al detalle en la mayoría de los proyectos (Pequeña o mediana escala)
- Pero un sistema que funciona bien no es necesariamente seguro



- Los programas vulnerables pueden tener características extra no intencionadas, formas de uso para los cuales el programa no fue diseñado.
- Un programa seguro es aquel que cumple con la funcionalidad esperada (dada en los requerimientos) y nada más.



- Cuando se desarrolla un software generalmente el equipo de desarrollo piensa en errores que los usuarios pueden llegar a hacer, pero no solo eso, errores y malos usos no intencionados, por lo general no suelen considerar a usuarios malintencionados.
- Siempre se tiene que pensar en un rival (Equipo rojo), alguien que de manera intencionada y con habilidades para hacerlo, lo va a intentar hallar y explotar una vulnerabilidad.



- Se tiene que evitar la mentalidad de:
- “He trabajado con varios sistemas y con la seguridad mínima es más que suficiente”
- “Nunca me han atacado y no creo que lo hagan”
- “A mí no me va a pasar eso, ¿por qué debería preocuparme?”.



Características de seguridad diferentes a características seguras



- Los programadores cuando quieren llevar a cabo seguridad, a menudo piensan en las características de la misma como:
 - Cifrado de información, reglas de contraseñas, control de accesos, entre otros.
- Para que un programa sea seguro, todos niveles deben ser seguros, no únicamente segmentos del mismo que hagan alusión a la propia seguridad.

Características de seguridad diferentes a características seguras



- Hay muchas formas de que la seguridad falle y que esto pueda poner en riesgo toda la seguridad del sistema en muchas formas.
- Por lo general, esto sucede por características defectuosas que no son parte de la seguridad, como el mal manejo de memoria
- El mal manejo de memoria puede abrir puertas para ejecutar código malicioso.

Características de seguridad diferentes a características seguras



- Imagina que estás desarrollando un programa en C para la computadora que abre la puerta del área de cómputo de la universidad. El sistema tiene una función rutinaria para registrar la asistencia de los estudiantes pidiendo su **nombre**.
- En la memoria RAM del servidor, el espacio se organiza de la siguiente manera:
 - Una variable llamada **nombre** con espacio exacto para 10 caracteres
 - Justo al lado en la memoria, una variable booleana de control llamada **es_administrador** (que por defecto vale 0 o Falso).

Características de seguridad diferentes a características seguras



- **Caso 1: El funcionamiento normal**
- Un **estudiante** llamado "Carlos" (6 letras) se registra en la terminal.
- El programa usa una función de copia insegura (strcpy) para guardar "Carlos" en la variable **nombre**.
- Como 6 caracteres caben perfectamente en nuestro límite de 10, la memoria luce ordenada:
- [C A R L O S _ _ _ _] (Buffer de nombre)
- [0] (La variable **es_administrador** sigue intacta en **Falso**, la puerta no se abre).

Características de seguridad diferentes a características seguras



- **Caso 2: El error de manejo de memoria**
- Un atacante se acerca a la terminal, pero en lugar de escribir su nombre, ingresa una cadena masiva de **25 caracteres** (por ejemplo: AAAA_SOY_ADMIN_123456789).
- La función de copia del programa toma el texto del atacante y el **programa jamás verifica si el texto cabe en los 10 bytes asignados.**
- **Paso 1: El desbordamiento**
- Los primeros 10 caracteres llenan el buffer de la variable **nombre**. Pero como el texto tiene 15 caracteres más, estos **continúan escribiéndose a la fuerza hacia la derecha**, invadiendo las casillas vecinas de la memoria RAM.
- Los caracteres extra del atacante "pisan" y sobrescriben el valor exacto donde vivía la variable **es_administrador**.



- **Paso 2: La caída total de la seguridad**
- Como el texto del atacante contenía código o valores numéricos específicos en esa posición exacta, el valor de la variable **es_administrador** cambia mágicamente de 0 (Falso) a 1 (Verdadero).
- El programa evalúa la variable, ve que ahora es 1 y le otorga privilegios de root/administrador al atacante, abriendo la puerta del laboratorio, **sin haber robado ninguna contraseña ni burlado ningún cifrado.**

Características de seguridad diferentes a características seguras



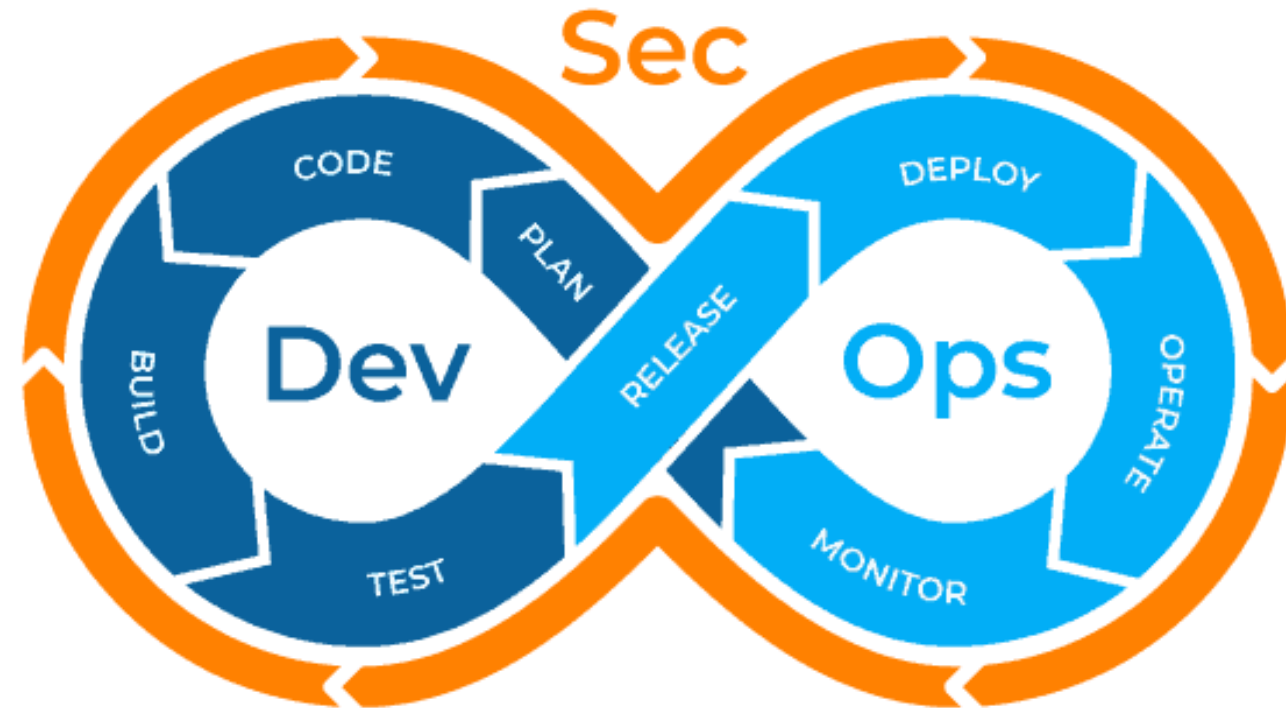
- Las características de seguridad son implementadas con la intención de que tienen que funcionar adecuadamente para que se mantenga la seguridad del programa.
- Sin embargo, las características que no tienen que ver con la seguridad directamente como lo pudimos ver con el ejemplo anterior, usualmente no reciben mucha atención, aunque pueda ser una entrada crítica de gente maliciosa.
- Los programadores comenten con frecuencia este tipo de errores, dejando de pensar en la seguridad cuando en realidad ese debería ser uno de sus principales enfoques.



- Realizar una investigación sobre DevSecOps de una cuartilla donde se consideré lo siguiente:
 - ¿Qué es DevSecOps?
 - ¿Cuál es su importancia?
 - ¿Qué prácticas incluye?
 - ¿Cómo se relaciona con la experiencia educativa?



- **¿Qué es DevSecOps?**
 - Es un **framework** que ayuda a los equipos de trabajo a centralizar las actividades tanto de desarrollo como de operaciones como un proceso individual
- **¿Para que sirve?**
 - Para armonizar el proceso de desarrollo de software con la seguridad del mismo, considerándolo en todas las etapas de desarrollo de vida de vida del software





• Problema real de la seguridad

Incentivos

Se premia al equipo de desarrollo por lanzar nuevas funcionalidades rápido, pero la seguridad es más como un mal necesario, hacer las cosas bien rara vez se ha premiado.

Complejidad actual

Antes todo era más reducido, solo había un servidor, hoy hay aplicaciones con microservicios, existen múltiples bases de datos, nubes, etc.



- **Fracaso de la seguridad tradicional**

Seguridad tradicional	Seguridad moderna (Zero Trust)
Confía en usuarios dentro de la red	Desconfía de todos, dentro y fuera de la red
Revisa las credenciales solo al ingresar	Verifica constantemente la identidad y el dispositivo utilizado
Da acceso total a la red interna	Da acceso mínimo y restringido solo a lo necesario



Vulnerabilidades



- Una vulnerabilidad es un defecto del sistema que permite a un atacante eludir las medidas de seguridad implementadas.
- Los errores y fallos no son algo nuevo en el mundo del software, un sistema extenso y complejo puede contener una gran cantidad de fallos y estos pueden ser explotados en ocasiones por usuarios malintencionados.
- Los problemas de seguridad causados por vulnerabilidades de software resultan especialmente preocupantes cuando se compromete información privada.



- Estas vulnerabilidades del software se pueden detectar mediante ejercicios como la auditoría de código, pentesting y herramientas automatizadas.



Auditoria de software



Pentesting



**Herramientas
automatizadas**



- La forma de prevenir las vulnerabilidades se hace siguiendo buenas prácticas y teniendo conciencia de la seguridad integral del proyecto que se este elaborando.





- Posible clasificación de alto nivel
 - Defectos genéricos
 - Defectos de contexto



- Son problemas que pueden ocurrir en cualquier programa hechos en un lenguaje.
- Pueden ser problemas específicos del lenguaje. Por ejemplo, en el caso de C es propenso a vulnerabilidades de memoria y por esto mismo a ataques de buffer overflow.
- Las herramientas de análisis estático de código son buenas para detectar este tipo de problemas.



- Requiere conocimiento semántico del problema que intenta resolver el programa
- Por ejemplo, en un programa que maneja tarjetas de crédito debe cuidarse que nunca sea visible el número completo de tarjeta
- Los defectos de este nivel pueden no ser sólo de codificación, sino también de diseño, por ejemplo tener un módulo de descarga de archivos adjuntos de correo y ejecuta el código que contienen.
- Este tipo de defectos requieren de configuración en una herramienta de análisis estático de código



- **Validación de entrada y representación:** Inyecciones
- **Abuso de API:** No seguir contrato
- **Características de seguridad:** Información sensible visible
- **Manejo de errores:** Dejar errores visibles
- **Calidad del código:** Código fácilmente estrésable
- **Encapsulación:** Mal manejo de la privacidad



Ingresa al siguiente enlace:

<https://portswigger.net/web-security>

1. Crea una cuenta de usuario
2. Da clic a la pestaña de “Academia”
3. Dirígete a la parte inferior de la interfaz hasta hallar el apartado de “Todos los tópicos”
4. Haz los ejercicios: **“Control de acceso”, “Autenticación” e “Inyección de SQL”**.
5. En un documento de PDF coloca paso a paso y con capturas de pantalla, como llevaste a cabo cada ejercicio y una breve descripción del proceso.



Modelado de amenazas



- Una amenaza es el objetivo de un atacante, para otros autores una amenaza es el atacante mismo
- La idea del modelado es entender posibles amenazas de seguridad del sistema, determinar riesgos y establecer medidas de mitigación apropiadas
- Permite elevar la conciencia de la organización hacia la seguridad



- El modelado de amenazas se encuentra en la etapa de diseño del software
- Lo que significa que se debe realizar antes de las revisiones de código o pruebas de seguridad
- Es probablemente la actividad de desarrollo más importante de seguridad ya que encamina todo el desarrollo
- Los defectos potenciales se localizan temprano y pueden resolverse rápido de forma barata



- En este curso se verá el método propuesto por el ciclo de vida de desarrollo seguro (SDL) de Microsoft
- Se utiliza como base la clasificación de riesgos STRIDE explicada más adelante
- También se utiliza como apoyo diagramas de diseño como diagramas de flujo de datos (DFD) y diagramas de contexto



- Contribuye al proceso de manejo de riesgos porque las amenazas al software e infraestructura son riesgos para el usuario y el ambiente donde se despliega el software
- Permite descubrir amenazas antes de que el sistema empiece a codificarse
- Revalida el diseño y arquitectura al hacer que el equipo de desarrollo revise de nuevo el diseño
- Contribuye a reducir la superficie de ataque
- Ayuda a guiar el proceso de revisión de código
- Guía el proceso de pentesting
- Clarifica la selección de medidas de mitigación



- La salida principal de un proceso de modelado de amenazas es un documento que describe la información de trasfondo de la aplicación
- El documento define el modelo de alto nivel de la aplicación, usando diagramas de flujo de datos



- Lo más importante a identificar en dichos diagramas es:
 - **Escenarios de uso:** Usos generales del usuario, configuración de despliegue
 - **Dependencias externas:** Producto, componentes o servicios de los cuales depende el sistema
 - **Datos intercambiados entre elementos**
 - **Límites de privilegios:** Establecen cambios en los privilegios ya sea hacía arriba o hacía abajo



- **Preparar:** Los diseñadores del sistema modelan mediante DFDs el flujo de datos
- **Analizar:** Se analizan los modelos para localizar posibles amenazas
- **Determinar medidas de mitigación:** Se determina para cada amenaza una forma de mitigarla con sus posibles técnicas y tecnologías asociadas
- **Implementar medidas de mitigación**



- Al seguir un procedimiento de modelado de amenazas desde la **etapa de diseño, se esta favoreciendo una aproximación proactiva de seguridad**
- Esto en contraposición a una aproximación reactiva, donde **se reacciona hasta que hay un problema**

“Prevenir es mejor que lamentar”



- Es la etapa de mayor importancia, si se realiza mal el resto del modelado estará mal
- En este curso se utilizarán diagramas de flujos de datos (DFDs) que muestra la comunicación entre procesos del sistema y entidades externas como usuarios u otros sistemas
- Los DFDs pueden realizarse por niveles, al nivel más general (nivel 0) se le conoce como **diagrama de contexto**
- Se pueden realizar tantos niveles como haga falta dependiendo de la complejidad del sistema
- En este curso sólo se tratarán **diagramas de nivel 0 y nivel 1**



- El sistema se identifica como un único proceso
- Se identifican las entidades externas y su interacción general con el sistema a través de flujos de datos
- Se identifican límites de privilegios
- Es importante numerar cada componente
- Cada flujo se etiqueta con un verbo o bien un sustantivo, es usual encontrar verbos CRUD (Create, Read, Update y Delete) pero se debe preferir aquella etiqueta que describa mejor los datos que se transfieren

Símbolos utilizados en diagramas de flujo de datos



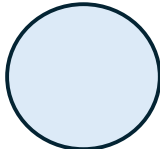
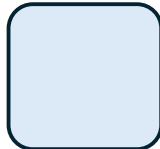
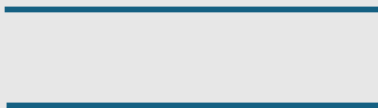
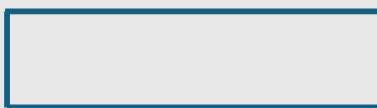
Componente	Símbolos de Yourdon y Coad	Símbolos de Gane y Sarson
Entidad externa		
Proceso		
Base de datos		
Flujo de datos		

Diagrama de contexto

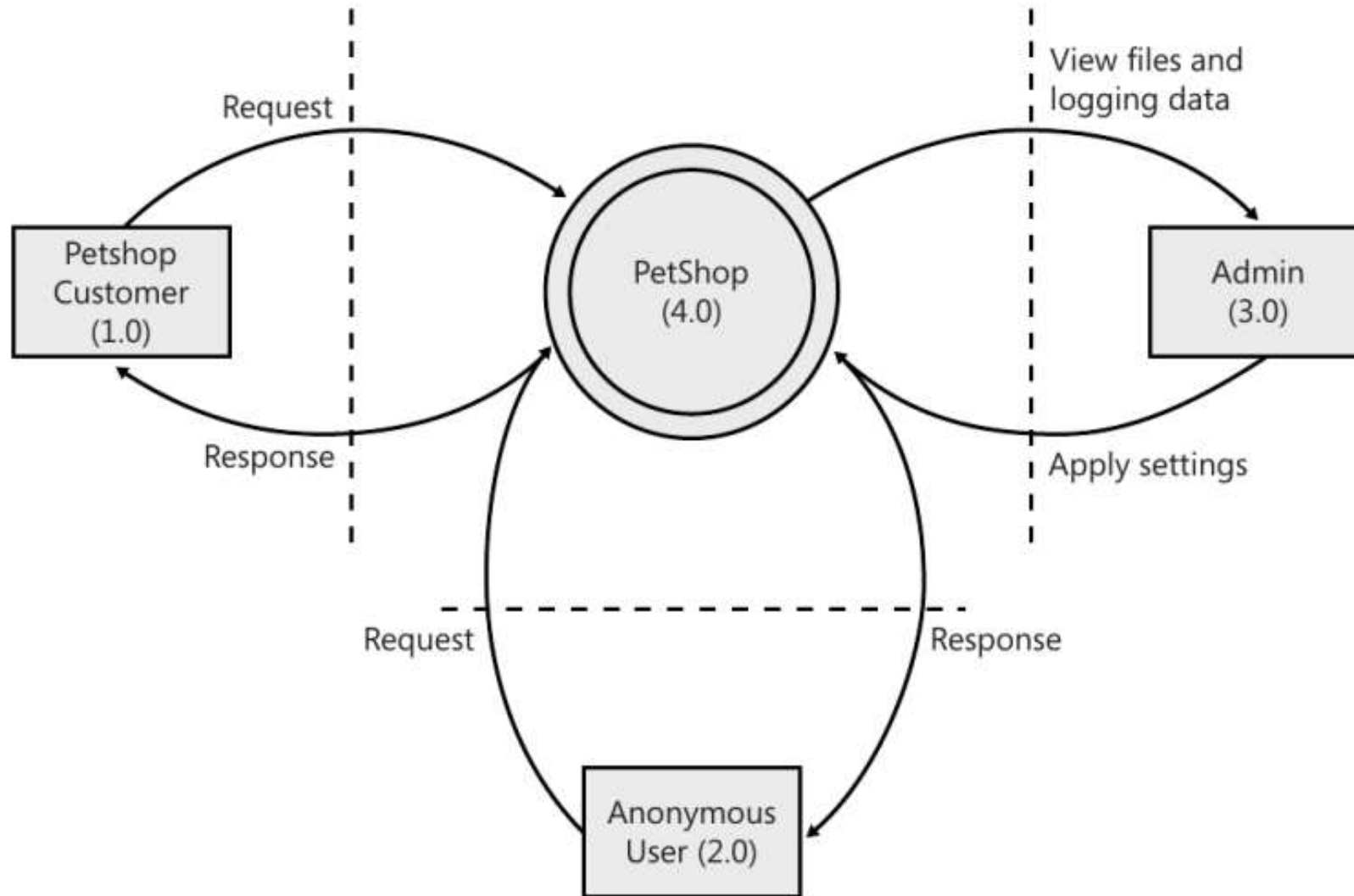


Diagrama de nivel 1



- El sistema general se descompone en subsistemas
- A este nivel se introducen las bases de datos con la que los procesos interactúan
- Es posible que sigan apareciendo procesos complejos, para cada proceso complejo se debe bajar de nuevo el nivel y hacer un DFD correspondiente
- En este curso solo se verá hasta el nivel 1

Diagrama de nivel 1

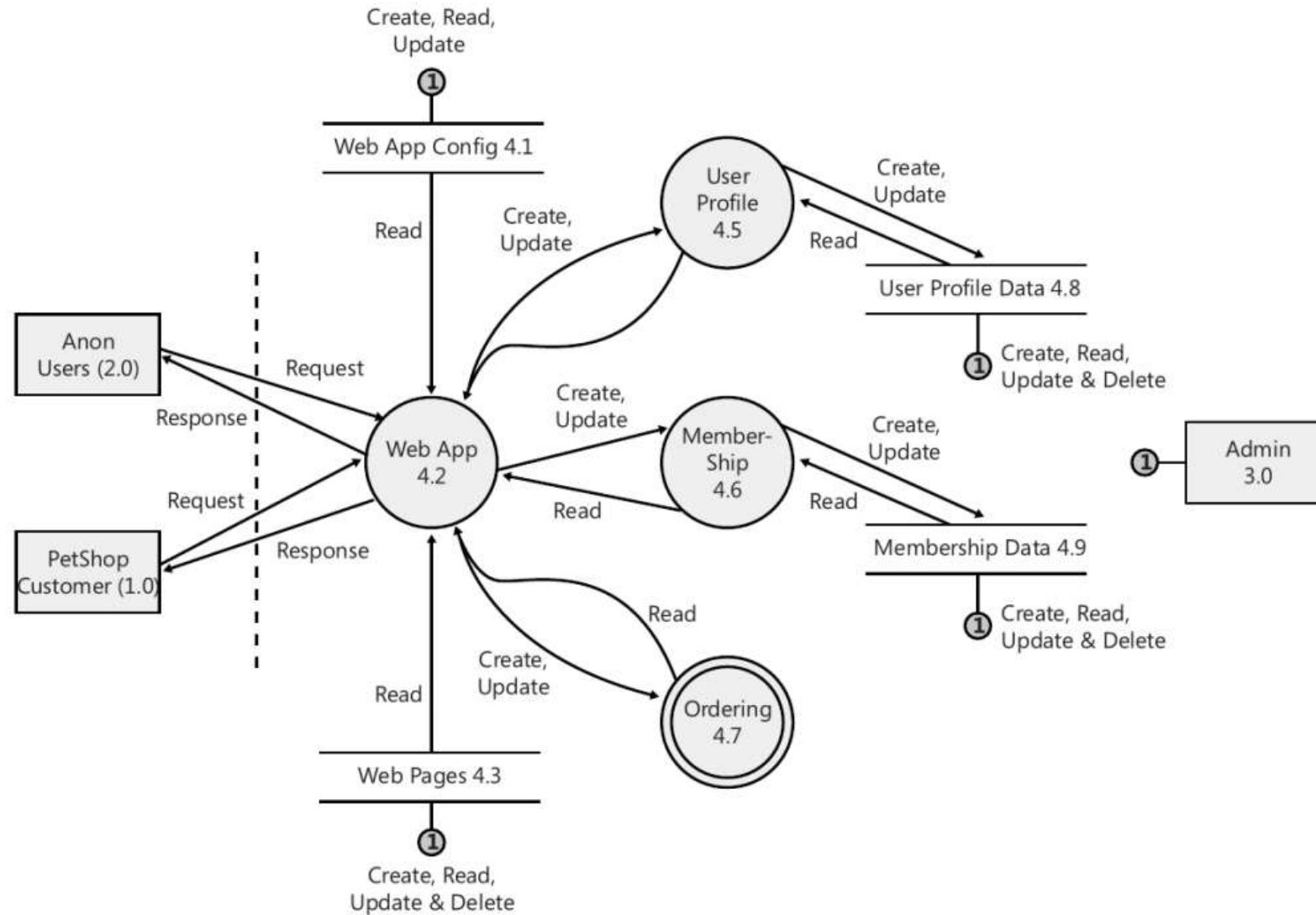
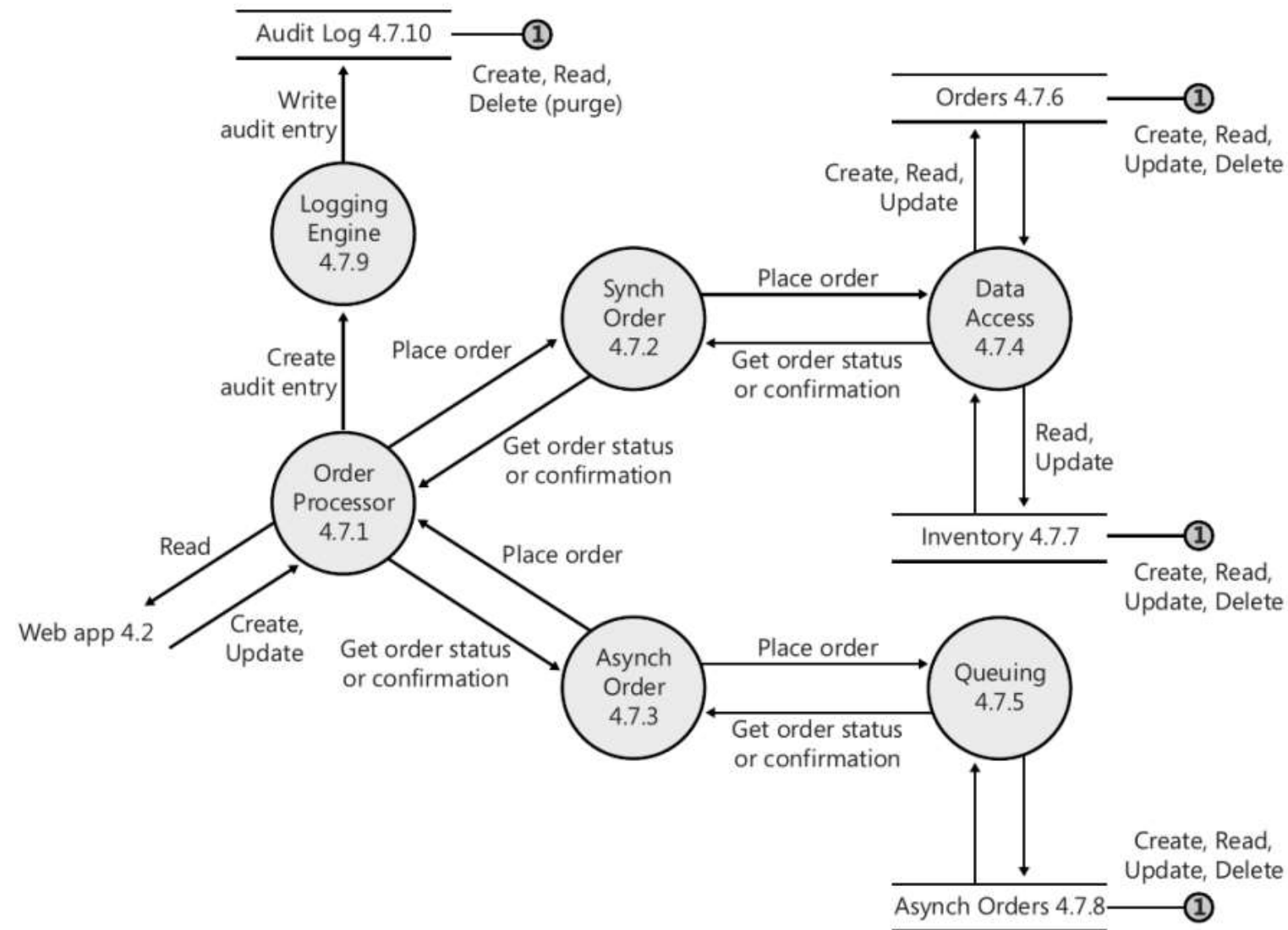


Diagrama de nivel 2





Determina el DFD de nivel 0 y de nivel 1 para el siguiente ejercicio:

Imagina que vamos a diseñar una app muy básica para escuchar música en el teléfono. Los elementos clave son:

- **El Oyente o Usuario:** Abre la app en su teléfono, busca una canción, le da play y escucha la música.
- **El Artista o Proveedor:** Sube nuevas canciones y portadas de álbumes a la plataforma para que los usuarios las puedan escuchar.



- La plataforma digital de una tienda de ropa en línea busca automatizar sus procesos de venta, empaque y control administrativo para mejorar su eficiencia operativa.
- El sistema interactúa de manera directa con tres perfiles de usuarios:
 - El **cliente**, quien ingresa a la plataforma para buscar prendas, armar su carrito de compras y realizar el pago.
 - El **almacenista**, encargado de recibir las listas de los productos vendidos para empacarlos y confirmar su envío.
 - El **administrador**, quien se encarga de dar de alta nueva ropa, modificar precios y consultar los reportes de ventas.



Tradicional CIA (En español CID):

- Confidencialidad (Confidentiality)
- Integridad (Integrity)
- Disponibilidad (Availability)

¿Qué es y de dónde viene?



- La tríada de la CIA es un modelo común que constituye la base para el desarrollo de sistemas de seguridad.
- Se utilizan para encontrar vulnerabilidades y métodos para crear soluciones.
- Proviene de la evolución de la seguridad de la información en el Departamento de Defensa de los Estados Unidos (DoD) y estudios gubernamentales durante las décadas de 1970 y 1980.



- Las organizaciones trabajan a partir de un conjunto de secretos: datos de nomina, información médica, contraseñas, secretos industriales, etc.
- El accesos a esos secretos debe estar estrictamente restringido, de lo contrario la información filtrada puede causar graves repercusiones
- La confidencialidad es mantener esos secretos a salvo, solo accesibles por las personas o sistemas autorizados
- Una forma de mantener la confidencialidad es por ejemplo el cifrado



- Tiene que ver con mantener un estado esperado de la información, esto es, que la información no sea alterada o destruida
- Las alteraciones pueden suceder a través de transmisiones o bien directamente en disco
- La integridad puede ser comprometida no solo por atacantes, sino también por fallos de hardware o software
- Una forma de asegurarse de que la información se mantiene íntegra es mediante algoritmos de hash



- Se refiere a la capacidad que tiene el sistemas y sus servicios para mantener un estado de operación y así brindar servicio a clientes
- Las alteraciones en la disponibilidad pueden dar de baja el servicio o bien mermar su tiempo de respuesta
- Es de suma importancia ya que muchas organizaciones dependen del mantenimiento continuo en la operación
- La forma más común de ataque a la disponibilidad es un ataque DOS o DDOS
- Este es del tipo de ataques que es más difícil defenderse



- Otra clasificación más completa que CIA y que considera problemas actuales de seguridad
- Tiene los siguientes tipos:
 - Spoofing Identity
 - Tampering
 - Repudiation
 - Information Disclosure
 - Denial of Service
 - Elevation of Privilege

Spoofing Identity



- Las amenazas de este tipo le permiten al atacante hacerse pasar por algo o alguien más
- Como un servidor haciéndose pasar por Microsoft.com o código que se hace pasar por una biblioteca de sistema



- Involucran modificaciones maliciosas de datos o código
- Los datos o código manipulado puede estar en un medio volátil o no volátil o incluso en la red



- Un atacante realiza una amenaza de repudio al negar haber efectuado una acción que otras personas no pueden confirmar o contradecir
- Por ejemplo, un usuario realiza repudio cuando efectúa una operación ilegal en un sistema que no puede rastrear dicha operación prohibida
- El no repudio es la habilidad del sistema para tratar amenazas de repudio
- Para lograr no repudio se debe tener evidencia mediante logging y registros de eventos



- Involucra la exposición de información a individuos que se supone no deberían tener acceso a ella
- Incluye tanto el acceso a archivos como a datos que viajan a través de la red
- Una amenaza de este tipo si no se trata puede convertirse en una violación de privacidad si los datos en cuestión son confidenciales o personales



- Son ataques que deniegan o degradan el acceso a un servicio por parte de sus usuarios
- Es necesario protegerse de esta amenaza para mejorar la disponibilidad y confiabilidad del sistema

Elevation of Privilege



- Esta amenaza ocurre cuando un usuario o proceso gana un incremento en sus capacidades de permisos
- A menudo se da a través de un usuario anónimo que explota un bug de programación para tener acceso a privilegios de administrador



- Una vez tienes tus DFDs se necesitan listar sus elementos ya que necesitan ser protegidos contra ataques
- En la lista no deben incluirse procesos complejos sino los elementos internos que lo conforman

Identificar amenazas al sistema



Table 9-3 DFD Elements Within the Pet Shop 4.0 Application

DFD Element Type	DFD Item Numbers
External Entities	Pet Shop customer (1.0)
	Anonymous user (2.0)
	Administrator (3.0)
Processes	Web application (4.2)
	User profile (4.5)
	Membership (4.6)
	Order processor (4.7.1)
	Synchronous order processor (4.7.2)
	Asynchronous order processor (4.7.3)
	Data access component (4.7.4)
	Queuing component (4.7.5)
Data Stores	Auditing engine (4.7.9)
	Web application configuration data (4.1)
	Web pages (4.3)
	User profile data (4.8)
	Membership data (4.9)
	Orders data (4.7.6)
	Inventory data (4.7.7)
	Asynch orders data (4.7.8)
	Audit-log data (4.7.10)



- De la lista anterior se puede aplicar un proceso de reducción
- Si tienes dos o más elementos del mismo tipo (por ejemplo dos o más procesos) que se encuentran detrás del mismo límite de confianza se puede modelar el elemento como una misma entidad, siempre y cuando los elementos usen la misma tecnología y manejen datos similares
- Esta reducción puede quitar muchos flujos, en la mayoría de casos es posible reducir todos los flujos bidireccionales

Identificar amenazas al sistema



Table 9-4 Reduced DFD Elements Within Pet Shop 4.0

DFD Element Type	DFD Item Number
External Entities	Pet Shop customer (1.0)
	Anonymous user (2.0)
	Administrator (3.0)
Processes	Web application (4.2)
	User profile (4.5)
	Membership (4.6)
	Order processor (4.7.1)
	Sync/Async order processors (4.7.2 and 4.7.3)
	Data-access or queuing components (4.7.4 and 4.7.5)
	Auditing engine (4.7.9)
Data Stores	Web application configuration data (4.1)
	Web pages (4.3)
	User profile data (4.8)
	Membership data (4.9)
	Order and async orders data (4.7.6 and 4.7.8)
	Inventory data (4.7.7)
Data Flows (partial list)	Audit-log data (4.7.10)
	Web application reading configuration data (4.1→4.2)
	Web pages read by Web application (4.3→4.2)
	Anonymous user request/response (2.0→4.2→2.0)
	Pet Shop customer request/response (1.0→4.2→1.0)
	Admin reading, creating, updating Web application configuration data (3.0→4.1→3.0)
	Admin reading, creating, updating, deleting Web pages (3.0→4.3→3.0)
	Web application reading, creating, updating an order (4.2→4.7.1→4.2)

Identificar amenazas al sistema



- Una vez se tiene la lista anterior se puede aplicar STRIDE a cada elemento, a partir del siguiente mapeo
- Esencialmente todo lo que esta en el DFD es propenso a ser atacado y la naturaleza del ataque potencial es determinado por el tipo de elemento

Table 9-5 Mapping STRIDE to DFD Element Types

DFD Element Type	S	T	R	I	D	E
External Entity	X		X			
Data Flow		X		X	X	
Data Store		X	†	X	X	
Process	X	X	X	X	X	X



- El siguiente paso es asociar las amenazas con los elementos identificados

Table 9-7 Threats to the System

Threat Type (STRIDE)	DFD Item Numbers
Spoofing	External entities: (1.0), (2.0), (3.0) Processes: (4.2), (4.5), (4.6), (4.7.1), (4.7.2 and 4.7.3), (4.7.4 and 4.7.5), (4.7.9)
Tampering	Processes: (4.2), (4.5), (4.6), (4.7.1), (4.7.2 and 4.7.3), (4.7.4 and 4.7.5), (4.7.9) Data stores: (4.1), (4.3), (4.8), (4.9), (4.7.6 and 4.7.8), (4.7.7), (4.7.10) Data flows: (4.1→4.2), (4.3→4.2), (2.0→4.2→2.0), (1.0→4.2→1.0), (3.0→4.1→3.0), (3.0→4.3→3.0), (4.2→4.7.1→4.2)
Repudiation	External entities: (1.0), (2.0), (3.0) Data flow: (4.7.10)
Information disclosure	Processes: (4.2), (4.5), (4.6), (4.7.1), (4.7.2 and 4.7.3), (4.7.4 and 4.7.5), (4.7.9) Data stores: (4.1), (4.3), (4.8), (4.9), (4.7.6 and 4.7.8), (4.7.7), (4.7.10) Data flows: (4.1→4.2), (4.3→4.2), (2.0→4.2→2.0), (1.0→4.2→1.0), (3.0→4.1→3.0), (3.0→4.3→3.0), (4.2→4.7.1→4.2)



Table 9-7 Threats to the System

Threat Type (STRIDE)	DFD Item Numbers
DoS	Processes: (4.2), (4.5), (4.6), (4.7.1), (4.7.2 and 4.7.3), (4.7.4 and 4.7.5), (4.7.9) Data stores: (4.1), (4.3), (4.8), (4.9), (4.7.6 and 4.7.8), (4.7.7), (4.7.10) Data flows: (4.1→4.2), (4.3→4.2), (2.0→4.2→2.0), (1.0→4.2→1.0), (3.0→4.1→3.0), (3.0→4.3→3.0), (4.2→4.7.1→4.2)
EoP	Processes: (4.2), (4.5), (4.6), (4.7.1), (4.7.2 and 4.7.3), (4.7.4 and 4.7.5), (4.7.9)



- Estrategias de mitigación:
- No hagas nada: OK para amenazas de bajo riesgo
- Quita la característica: reduce riesgo a 0, balance entre funcionalidad y seguridad
- Deshabilita la característica: menos drástico que remover, puede aplicarse por rol de usuario
- Advierte al usuario: el usuario puede no entender los riesgos
- Mitiga la amenaza mediante tecnología: medida más común, por ejemplo usar autenticación o cifrado



Table 9-8 Mitigation Techniques Based on STRIDE Threat Type

Threat Type	Mitigation Technique
Spoofing	Authentication
Tampering	Integrity
Repudiation	Non-repudiation services
Information disclosure	Confidentiality
DoS	Availability
EoP	Authorization

♀



Table 9-9 Mitigation Technologies

Mitigation Technique	Mitigation Technology
Authentication	Authenticate principals: ■ Basic authentication ■ Digest authentication ■ Cookie authentication ■ Windows authentication (NTLM) ■ Kerberos authentication ■ PKI systems such as SSL/TLS and certificates ■ IPSec ■ Digitally signed packets Authenticate code or data: ■ Digital signatures ■ Message authentication codes ■ Hashes

Ejemplo de tecnologías por técnica



Integrity	■ Windows Vista Mandatory Integrity Controls ■ ACLs ■ Digital signatures ■ Message authentication codes
Non-repudiation services	■ Strong authentication ■ Secure auditing and logging ■ Digital signatures ■ Secure time-stamps ■ Trusted third parties
Confidentiality	■ ACLs ■ Encryption
Availability	■ ACLs ■ Filtering ■ Quota ■ Authorization
Authorization	■ ACLs ■ Group or role membership ■ Privilege ownership ■ Permissions



- El siguiente paso es tomar todas las amenazas del modelado, observar las técnicas de mitigación asociadas y determinar una tecnología de mitigación apropiada

Table 9-11 Defenses Used in Portions of Pet Shop 4.0

Asset	Asset Type	Example Threat	Example Mitigation
(1.0→4.2→1.0)	Data flow from Pet Shop user to Web application and back	I	SSL/TLS (also mitigates the tampering threat)
4.7.10	Audit log data store	T	ACL and MAC
4.7.1	Order processor process	T and E	ACL, MAC, and reduced process privilege



- A partir del ejercicio de la **tienda de ropa en línea**
- Identifica todos los **elementos del DFD en una tabla**, realiza las **reducciones convenientes**
- Asocia los elementos con los **tipos de amenazas STRIDE**
- Para cada elemento toma como base una **posible amenaza STRIDE que aplique** y propón **una tecnología de mitigación apropiada**
- La actividad deberá estar en un PDF